Blockchain Principles and Applications

Amir Mahdi Sadeghzadeh, PhD

Data and Network Security Lab (DNSL)
Trustworthy and Secure AI Lab (TSAIL)

Most slides have been adapted from ECE/COS 470, Pramod Viswanath, Princeton 2024 and CS251, Dan Boneh, Stanford 2023.

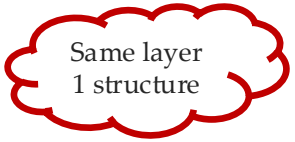
Layer 2 Scaling: Side Blockchains

Trust from Trusted Blockchain

- Trusted Blockchains
  - Bitcoin, Ethereum

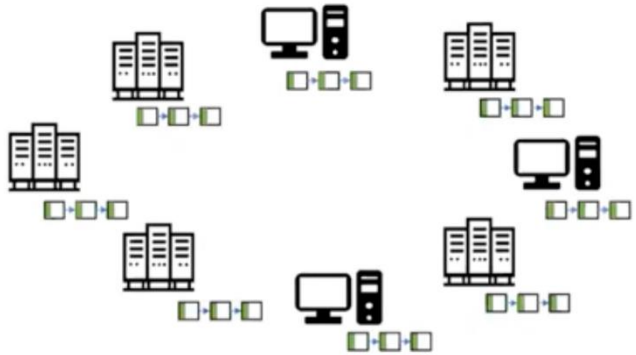
- New Blockchains
  - Ouroboros, Prism

- ***Side Blockchains***



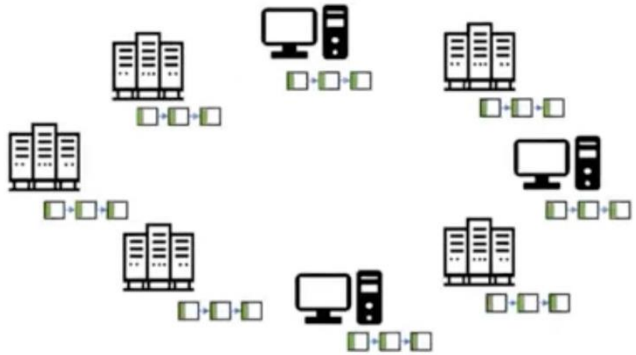
  - Derive trust from trusted blockchains
  - Commit hashes periodically

Prohibitive Storage Overhead



- Ledger maintained by a network of nodes

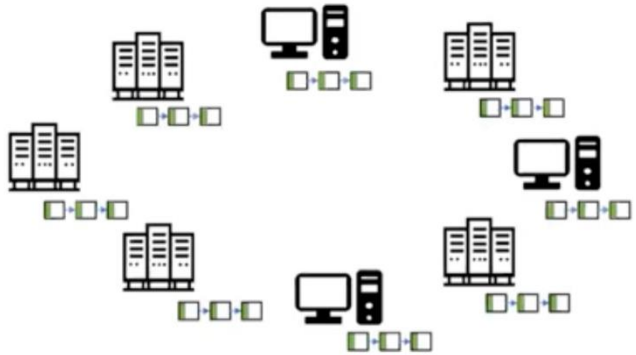
Prohibitive Storage Overhead



- Ledger maintained by a network of nodes
- Each node maintains a local copy of the ledger

Significant storage overhead

Prohibitive Storage Overhead



- Ledger maintained by a network of nodes
- Each node maintains a local copy of the ledger

- Bitcoin ledger size $\sim$ 365GB[1]
- Ethereum ledger size $\sim$ 990GB[2]

Significant storage overhead

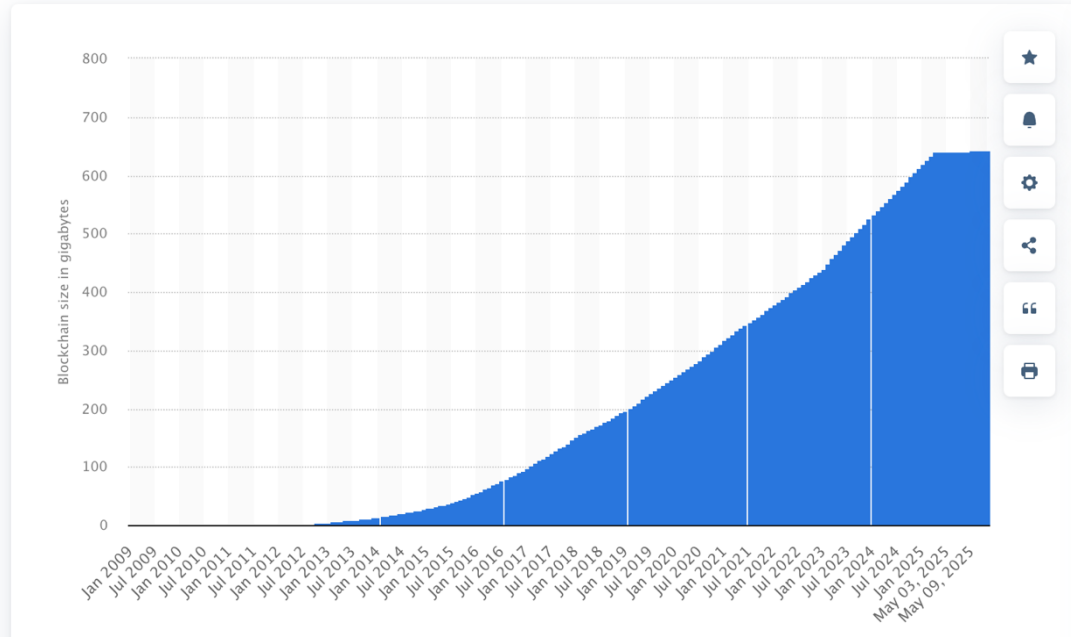
As of 10/5/2021, [1]https://www.blockchain.com/charts/blocks-size
[2]https://etherscan.io/chartsync/chaindefault

Bitcoin Leadger Size

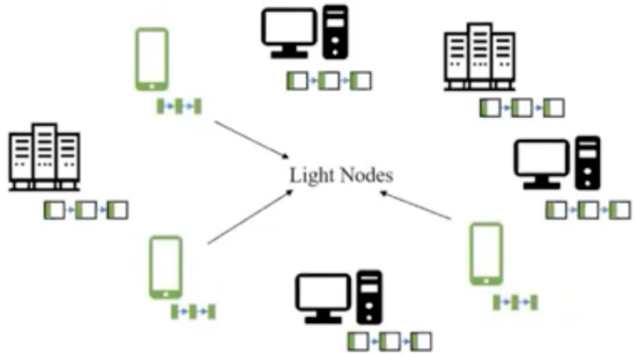
Finance & Insurance › Financial Instruments & Investments

Size of the Bitcoin blockchain from January 2009 to May 13, 2025

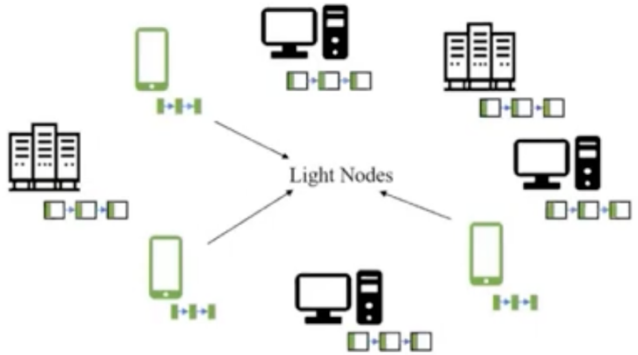
(in gigabytes)



Solution: Allowing Light Nodes

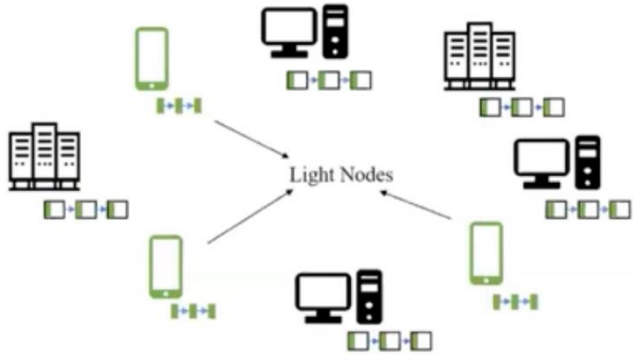


Solution: Allowing Light Nodes



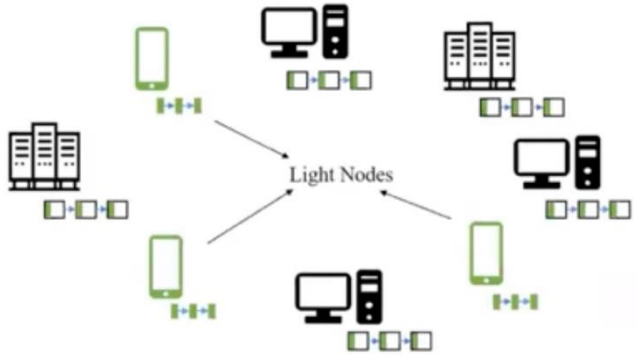
- Only store block headers
  (total size $\sim$ 1GB for Ethereum)

Solution: Allowing Light Nodes



- Only store block headers (total size $\sim$ 1GB for Ethereum)
- Can verify transaction inclusion in a block

Solution: Allowing Light Nodes



- Only store block headers (total size $\sim$ 1GB for Ethereum)
- Can verify transaction inclusion in a block
- Cannot verify transaction correctness $\rightarrow$ Rely on honest Full nodes for fraud notification

Nakamoto's solution:

Lite node

SPV node

Only wants to verify a transaction

Download chain of block headers from main node

header: nonce, parent.hash, Merkle root

periodically poll main nodes

update if there is a longer chain

When an SPV client wants to verify a transaction,

- it requests a Merkle proof from a full node.
  - The Merkle proof contains all the hashes along the path from the transaction to the Merkle root.
  - By hashing these values and comparing the result to the Merkle root in the block header, the SPV client can confirm that a specific transaction is indeed part of the block.

Limitations of Light Node Verification

Lite node

needs to be connected with a majority of honest nodes

This is a strong (networking) requirement

easily attacked via botnets

Verification via Fraud Proofs

Interactive verification mechanism

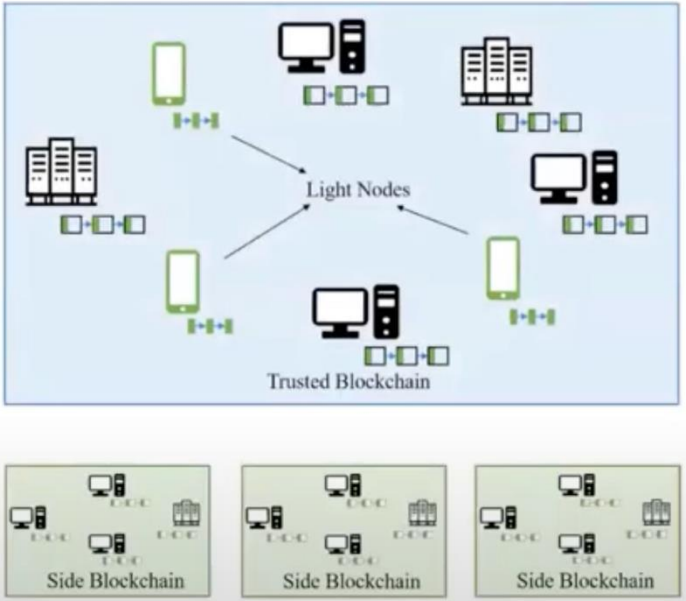
needs lite nodes to be connected to at least one honest node

Security depends on fraud proofs

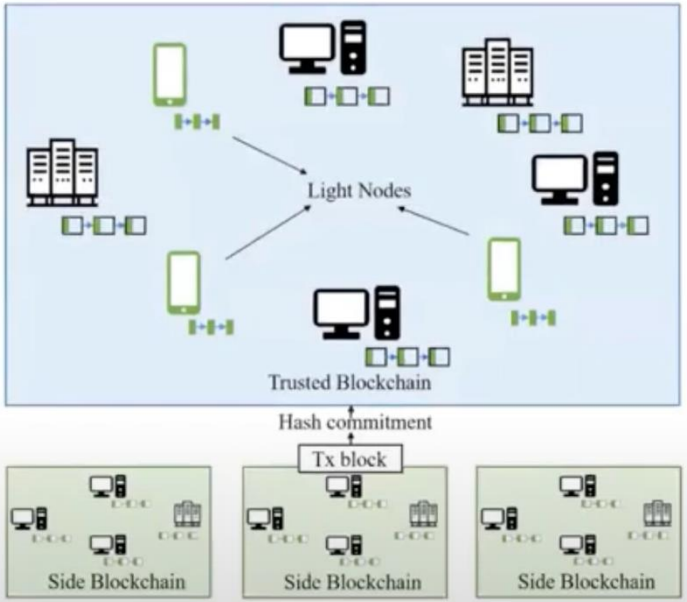
fraudulent activity is detected by full nodes that can furnish proof of fraud

proof of non-inclusivity in the Merkle tree

Running Side Blockchain



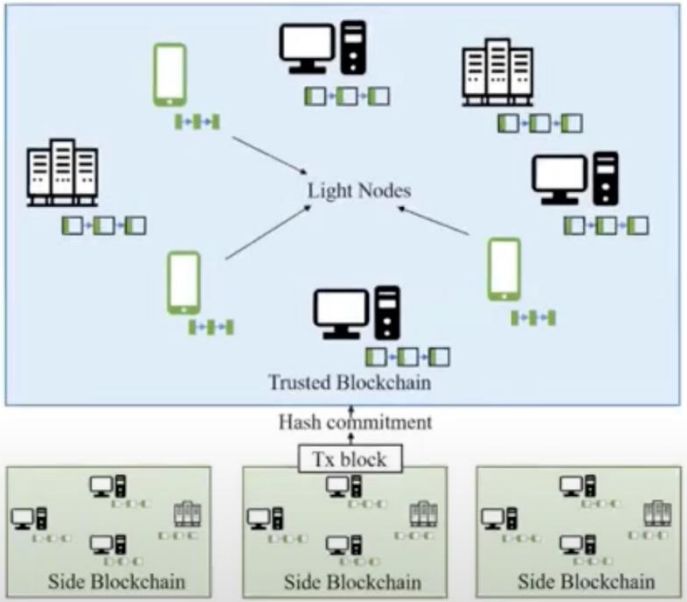
Running Side Blockchain



Side Blockchain nodes:

- Push hash commitment of their block to trusted blockchain

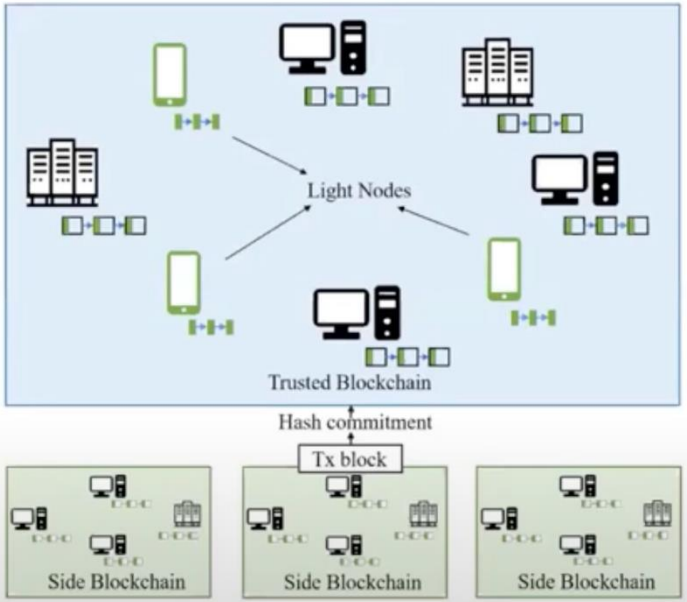
Running Side Blockchain



Side Blockchain nodes:

- Push hash commitment of their block to trusted blockchain
- Order of transactions same as hash order in trusted blockchain

Running Side Blockchain



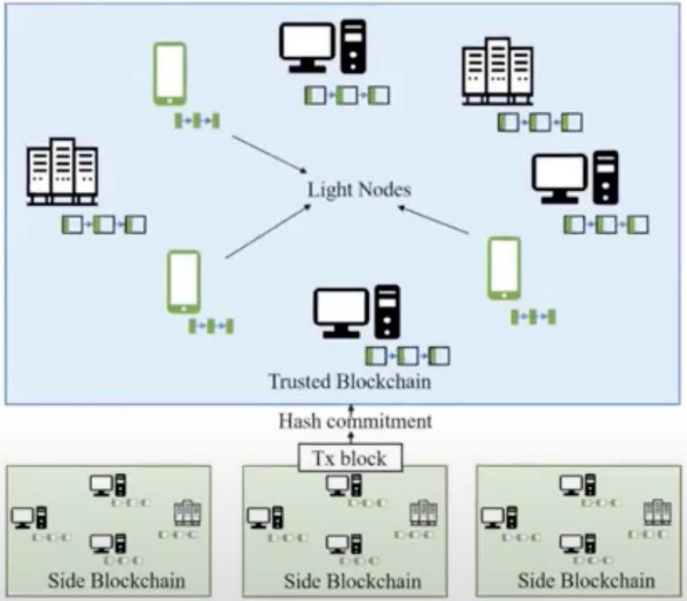
Side Blockchain nodes:

- Push hash commitment of their block to trusted blockchain
- Order of transactions same as hash order in trusted blockchain

Trusted Blockchain:

- Only store the hash of the side blockchain

Running Side Blockchain



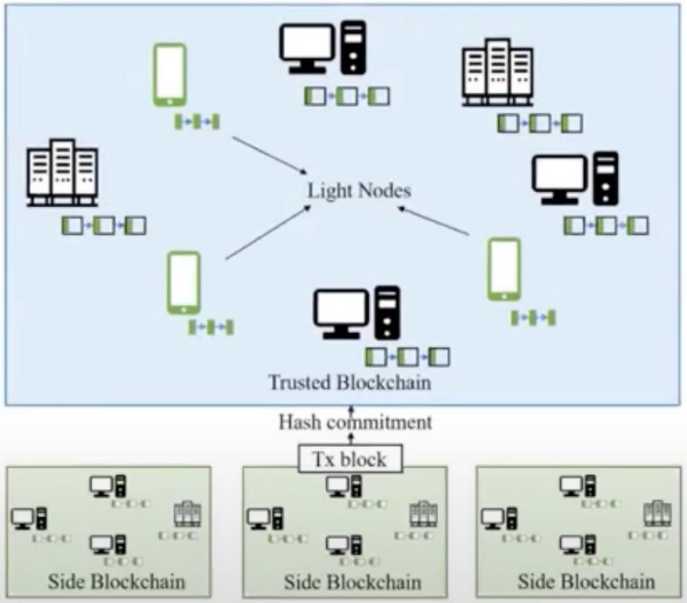
Side Blockchain nodes:

- Push hash commitment of their block to trusted blockchain
- Order of transactions same as hash order in trusted blockchain

Trusted Blockchain:

- Only store the hash of the side blockchain
- Side blockchains make commitments in parallel

Running Side Blockchain



Side Blockchain nodes:

- Push hash commitment of their block to trusted blockchain
- Order of transactions same as hash order in trusted blockchain

Trusted Blockchain:

- Only store the hash of the side blockchain
- Side blockchains make commitments in parallel
- Leads to higher transaction throughput

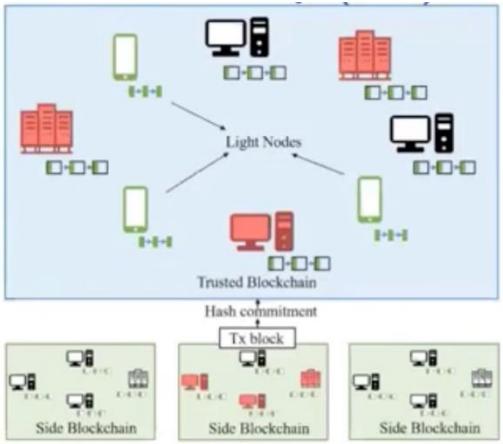
Data Availability Attack

A malicious block producer publishes a block header so that:

1. light nodes can check transaction inclusion, but

2. withholds a portion of the block (e.g., invalid transactions), so that it is impossible for honest full nodes to validate the block and generate the fraud proof.

Data Availability Attack



Light Nodes:

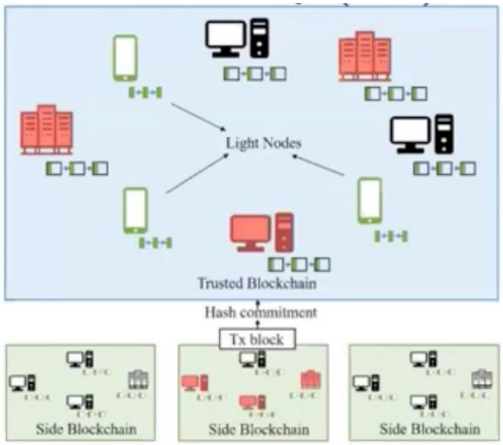
- Only store the header of each block
- Occur when there is a dishonest majority of full nodes

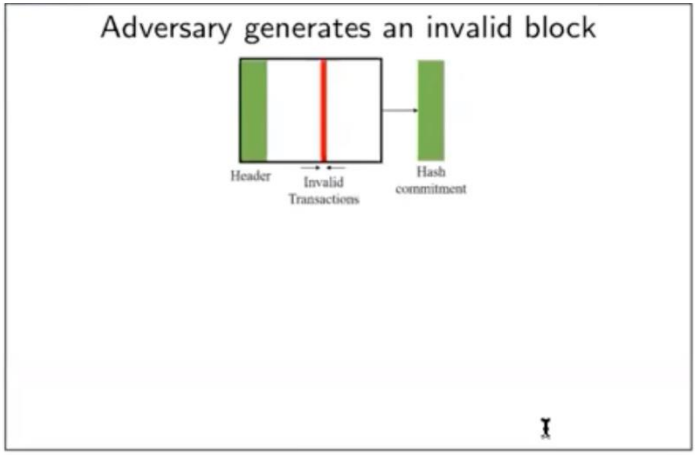
Trusted Blockchain:

- Only store the hash commitment of the side blockchain blocks
- Occur when there is a dishonest majority of side blockchain nodes

Both are vulnerable to Data Availability attacks

Data Availability Attack





Light Nodes:

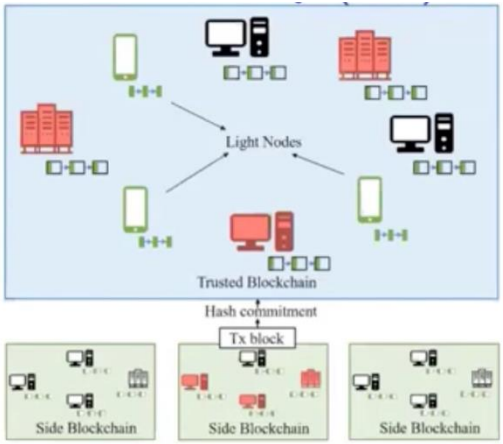
- Only store the header of each block
- Occur when there is a dishonest majority of full nodes

Trusted Blockchain:

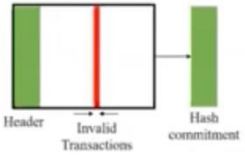
- Only store the hash commitment of the side blockchain blocks
- Occur when there is a dishonest majority of side blockchain nodes

Both are vulnerable to Data Availability attacks

Data Availability Attack



Adversary generates an invalid block



By hiding the invalid transactions:

- Adversarial full nodes trick light nodes to accept invalid header
- Adversarial side blockchain nodes commit invalid hash to trusted blockchain

Light Nodes:

- Only store the header of each block
- Occur when there is a dishonest majority of full nodes

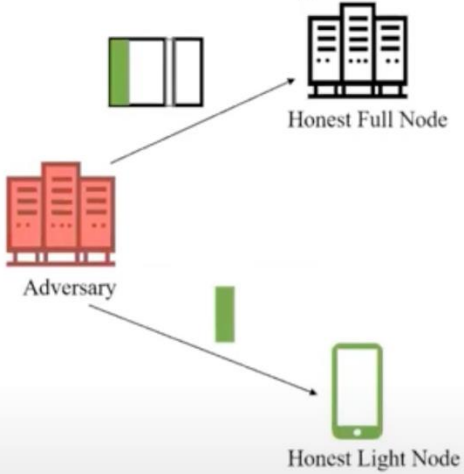
Trusted Blockchain:

- Only store the hash commitment of the side blockchain blocks
- Occur when there is a dishonest majority of side blockchain nodes

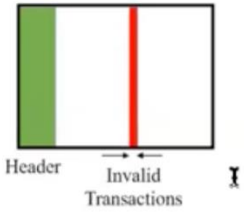
Both are vulnerable to Data Availability attacks

Data Availability Attack on Light Nodes

Systems with light nodes and a dishonest majority of full nodes are vulnerable to DA attacks [Al-Bassam '18], [Yu '19]



Adversary creates an invalid block



- Adversary: Provides block to Full node but hides invalid portion
  Provides header to Light node
- Honest Nodes: Cannot verify missing transactions → No fraud proof
- Light Nodes: No fraud proof → accept the header.

Conundrum for Honest Nodes

Honest full nodes are aware of the data unavailability
but there is no good way to prove it.

1. Best is to raise an alarm without a proof
this is problematic because the malicious block producer can release the hidden parts **after** hearing the alarm.

2. Due to network latency, other nodes may receive the missing part before receiving the alarm
thus, cannot distinguish who is prevaricating.

Need for Data Availability

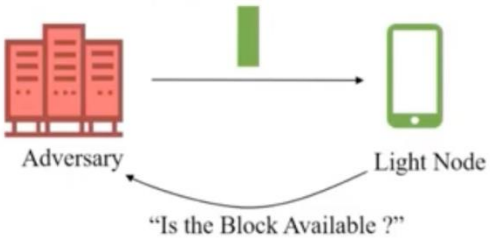
For fraud proofs to work:

light nodes must determine data availability by themselves.

Key question:

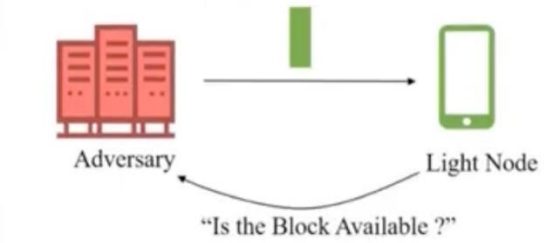
when a light node receives the header of some block, how can it verify that the content of that block is available to the network by downloading the least possible portion of the block?

Ensuring Data Availability



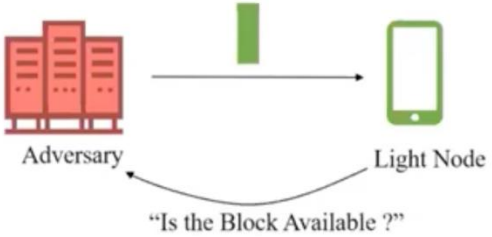
- Anonymously request/sample few random chunks of the block

Ensuring Data Availability



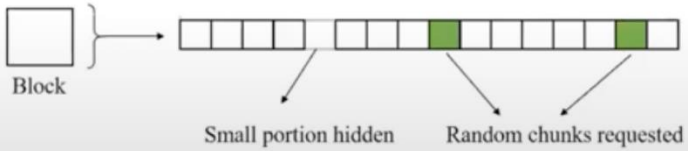
- Anonymously request/sample few random chunks of the block
- Adversary can hide a small portion

Ensuring Data Availability

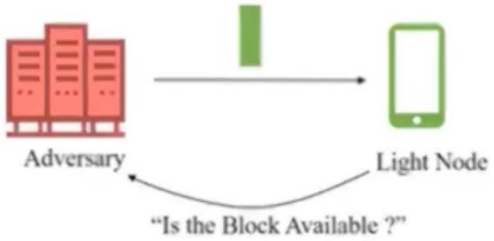


- Anonymously request/sample few random chunks of the block
- Adversary can hide a small portion

- Probability of failure: probability for a light node to accept a header whose corresponding block has hidden chunks



Ensuring Data Availability



- Anonymously request/sample few random chunks of the block
- Adversary can hide a small portion

- Probability of failure: probability for a light node to accept a header whose corresponding block has hidden chunks
  - ↳ probability of not requesting the hidden chunks

Probability of failure using 2 random samples:

$$\left(1-\frac{1}{16}\right)\left(1-\frac{1}{15}\right)=0.87$$

Can Erasure Coding improve the probability of failure?

Need to Encode the Block

A transaction is much smaller than a block

so, a malicious block producer only needs to hide a very small portion of a block.

Only way to detect such hiding by

the entire block is downloaded.

Encoding:

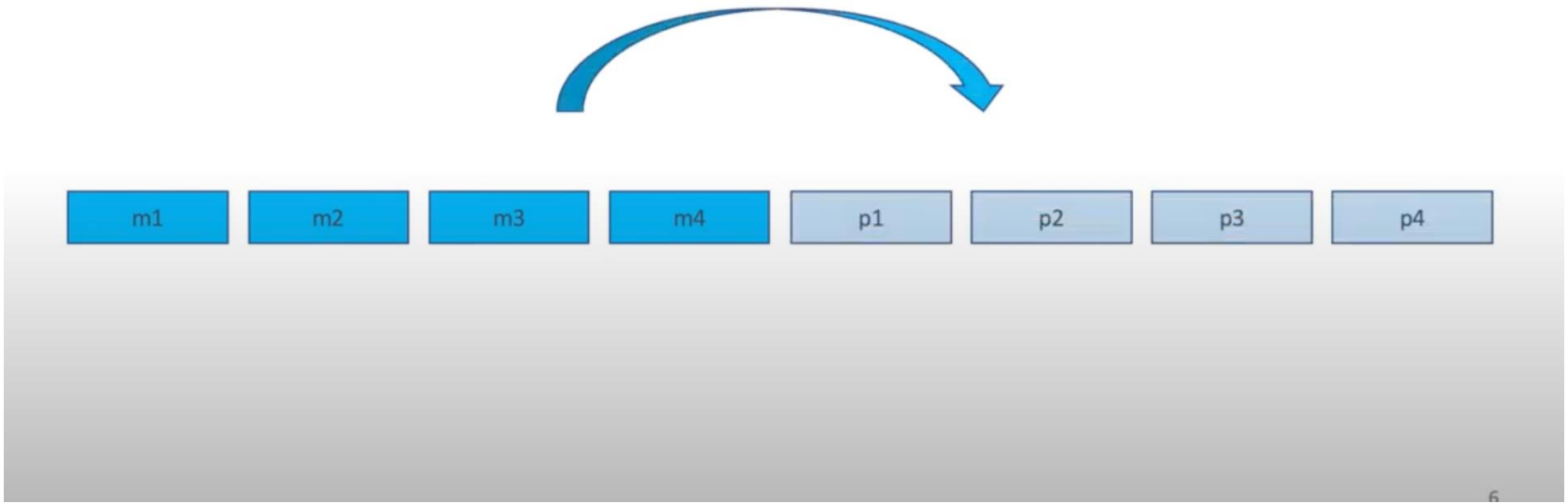
linear erasure codes

any small hiding will result in lot of data being unavailable

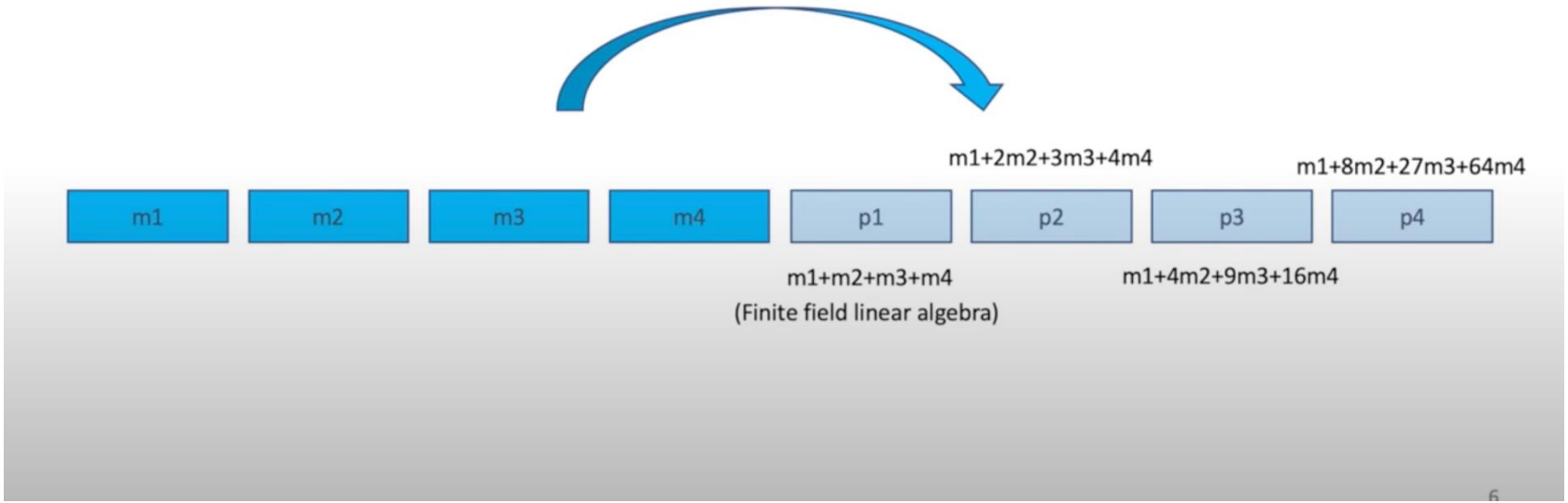
thus detected via random sampling

Introducing Erasure Coding

Introducing Erasure Coding

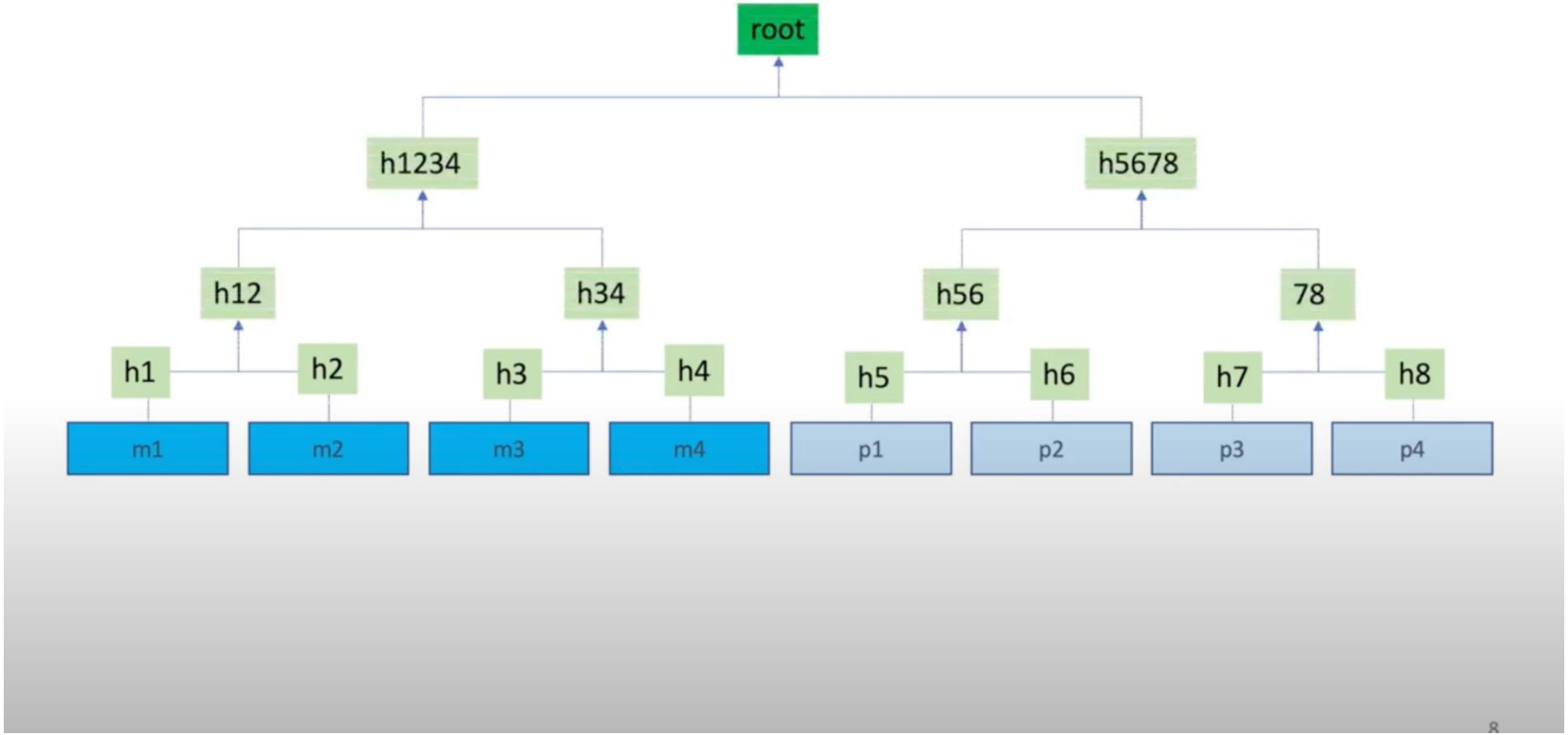


Introducing Erasure Coding

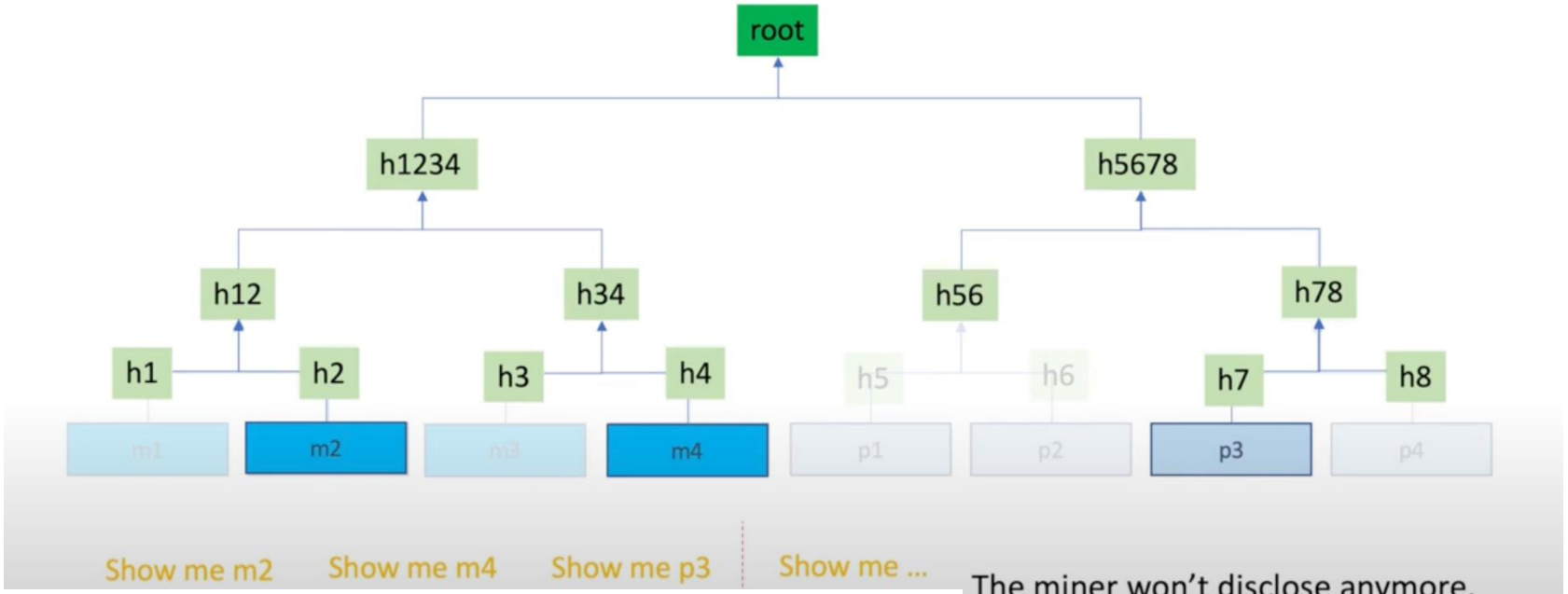


Introducing Erasure Coding

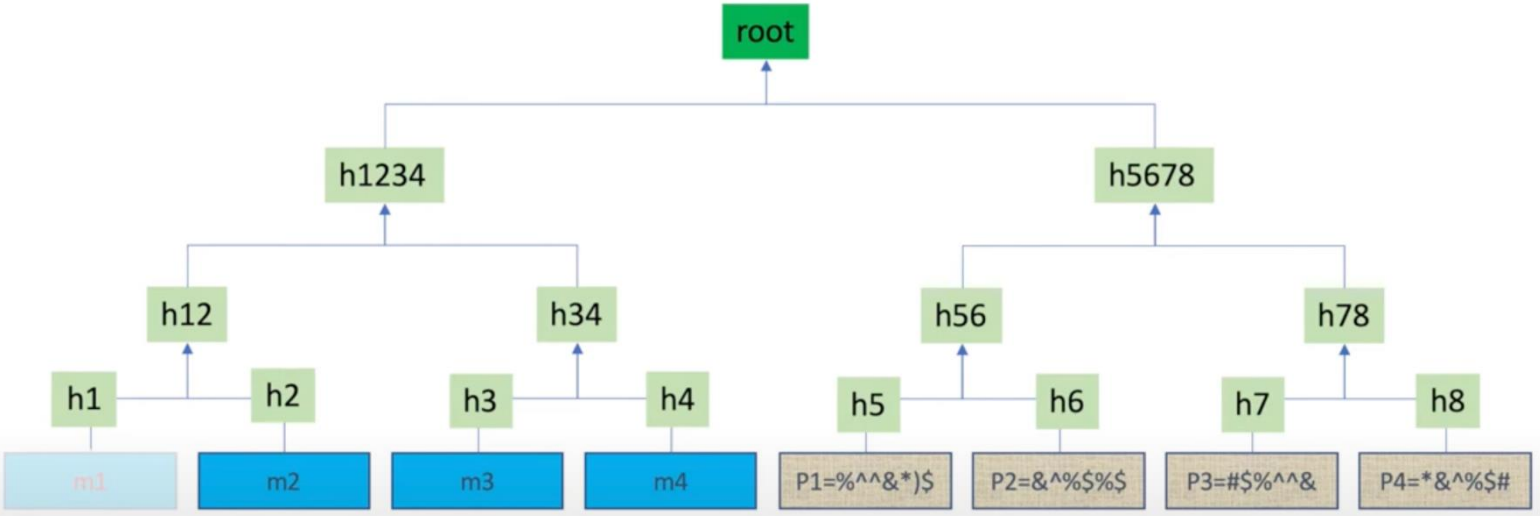
Merkle Tree of Erasure Coded Block



Merkle Tree of Erasure Coded Block

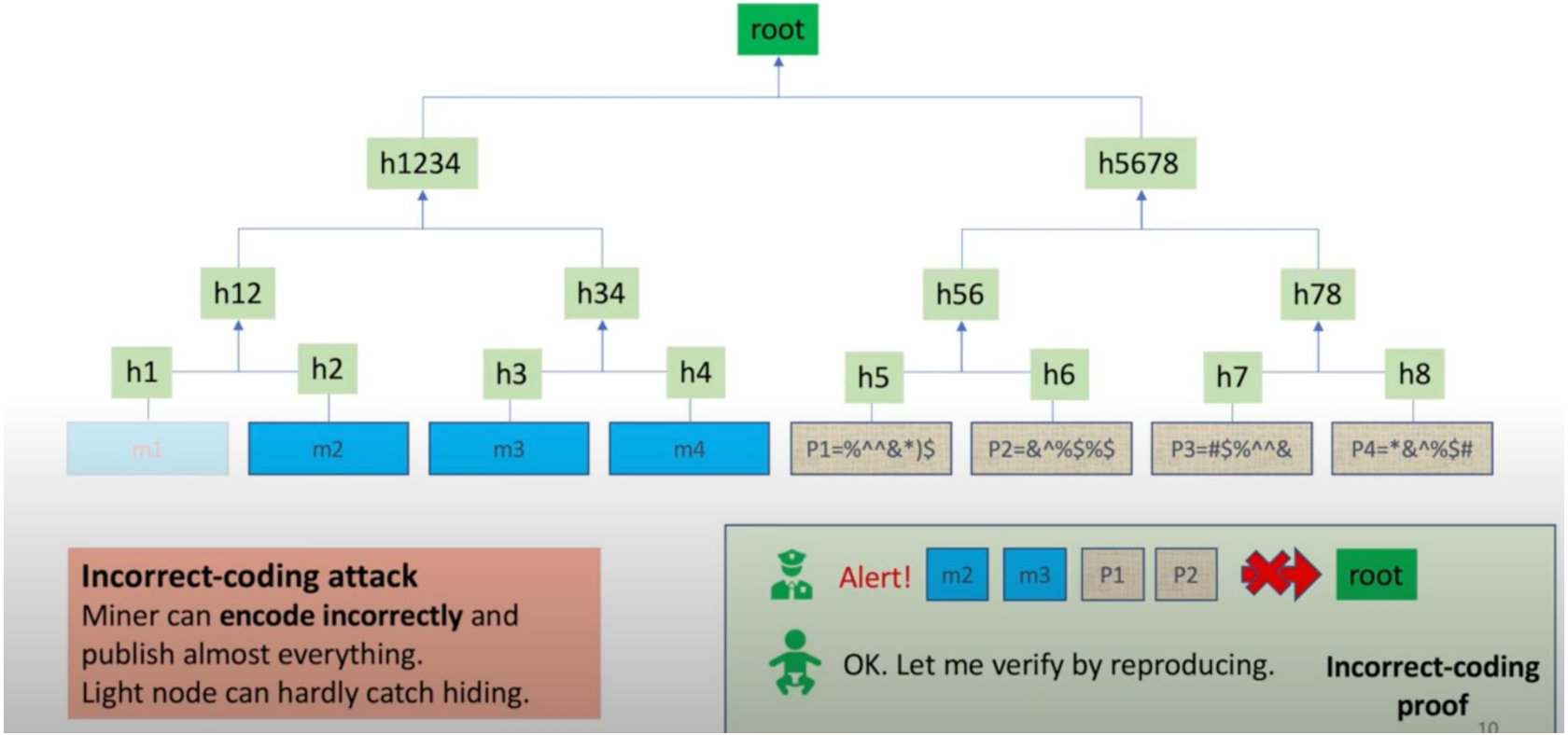


Incorrect-coding Attack

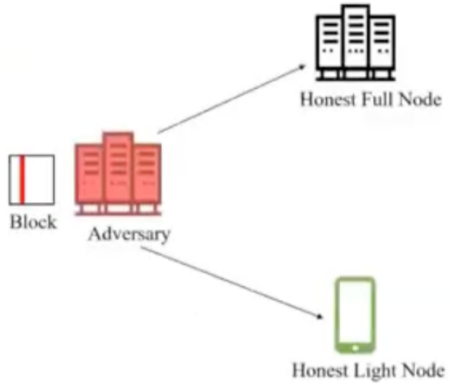


Incorrect-coding attack
Miner can **encode incorrectly** and publish almost everything.
Light node can hardly catch hiding.

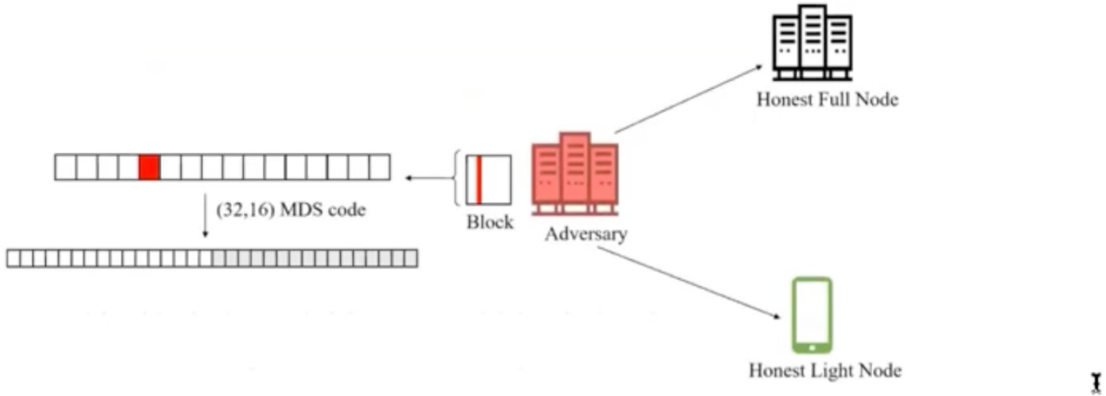
Incorrect-coding Attack



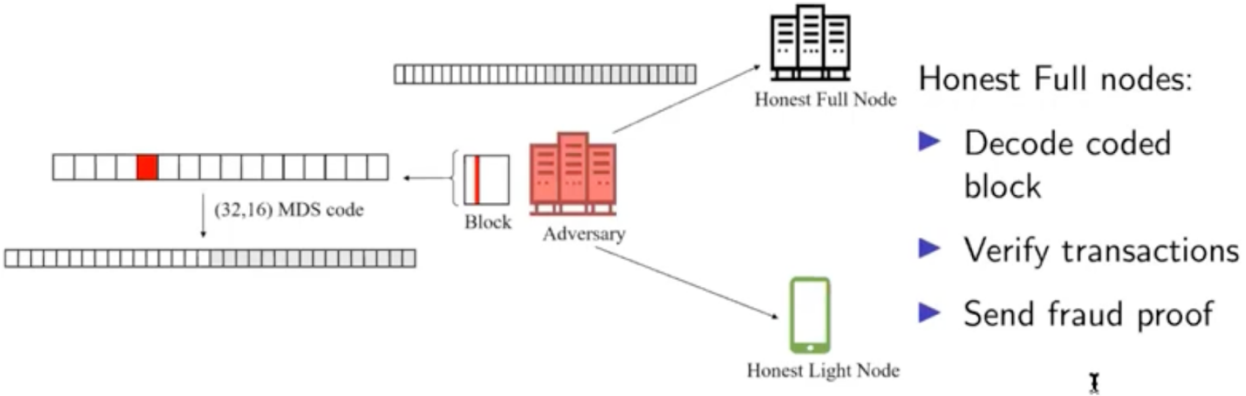
Erasure Coding to Improve the Probability of Failure?



Erasure Coding to Improve the Probability of Failure?



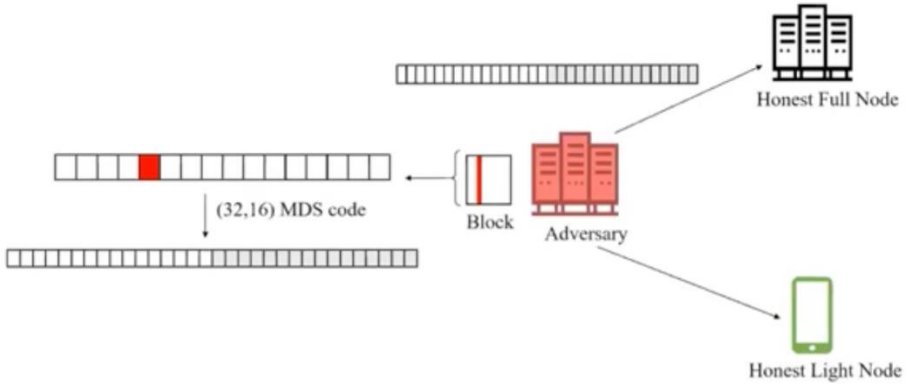
Erasure Coding to Improve the Probability of Failure?



Honest Full nodes:

- Decode coded block
- Verify transactions
- Send fraud proof

Erasure Coding to Improve the Probability of Failure?



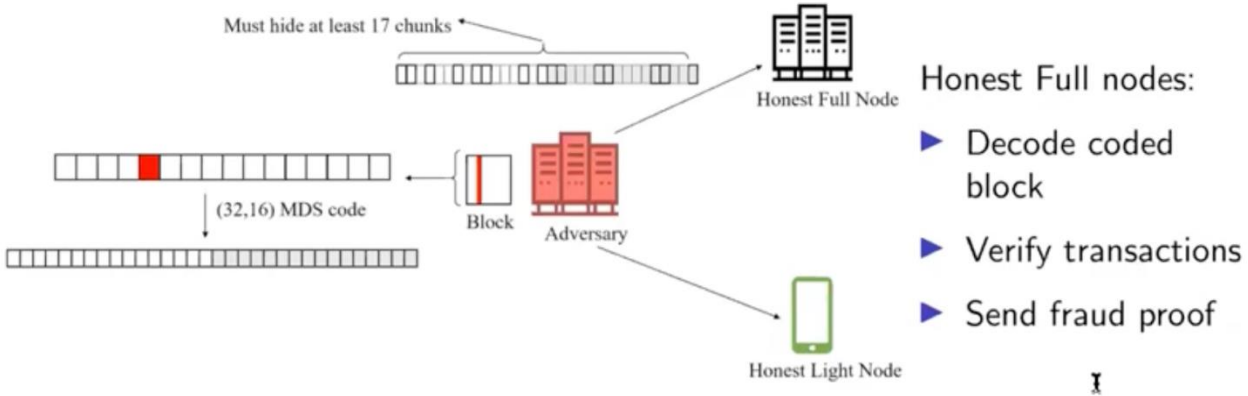
Honest Full nodes:

- Decode coded block
- Verify transactions
- Send fraud proof

Adversary:

- Want to prevent honest full nodes from sending fraud proof
- Must prevent honest full nodes from decoding back the original block
- Must hide more coded chunks

Erasure Coding to Improve the Probability of Failure?



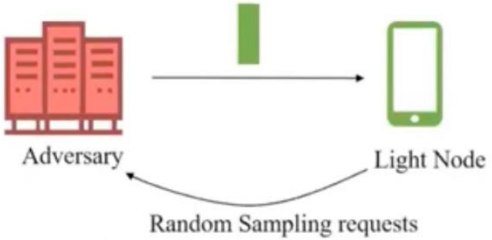
Honest Full nodes:

- Decode coded block
- Verify transactions
- Send fraud proof

Adversary:

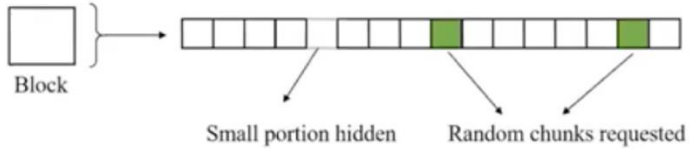
- Want to prevent honest full nodes from sending fraud proof
- Must prevent honest full nodes from decoding back the original block
- Must hide more coded chunks → Easier for light nodes to catch using random sampling

Erasure Coding to Improve the Probability of Failure?



- Light nodes request for samples of the coded block

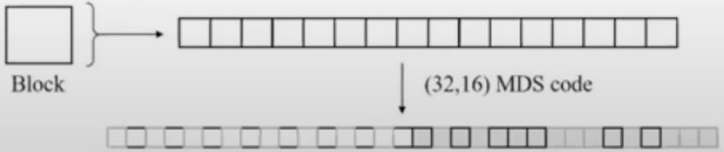
No coding:



Probability of failure using 2 random samples:

$$\left(1-\frac{1}{16}\right)\left(1-\frac{1}{15}\right)=0.87$$

Erasure coding:



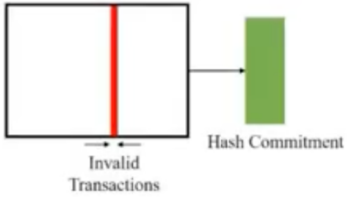
Probability of failure using 2 random samples:

$$\left(1-\frac{17}{32}\right)\left(1-\frac{17}{31}\right)=0.21$$

Data Availability Attack on Silde Blockchain

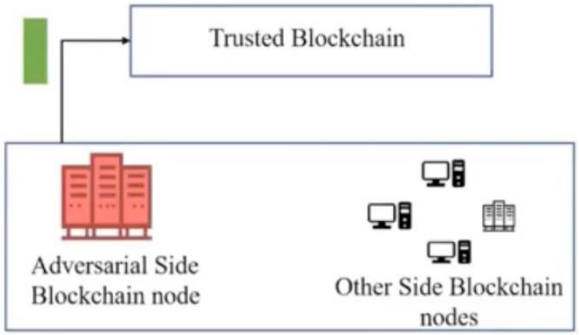
Side Blockchains with a majority of dishonest nodes are vulnerable to data availability attacks [Sheng '20]

Adversary creates an invalid block

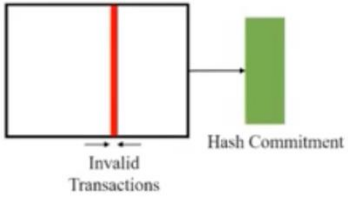


Data Availability Attack on Silde Blockchain

Side Blockchains with a majority of dishonest nodes are vulnerable to data availability attacks [Sheng '20]



Adversary creates an invalid block

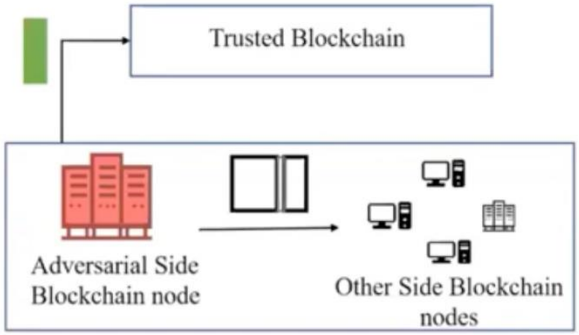


Adversarial Side blockchain node:

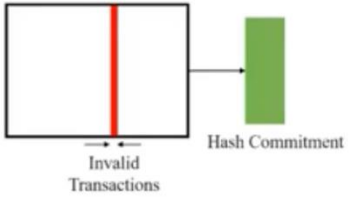
- Pushes hash commitment to the trusted blockchain

Data Availability Attack on Silde Blockchain

Side Blockchains with a majority of dishonest nodes are vulnerable to data availability attacks [Sheng '20]



Adversary creates an invalid block

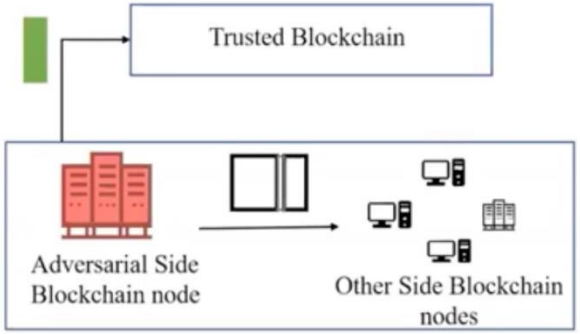


Adversarial Side blockchain node:

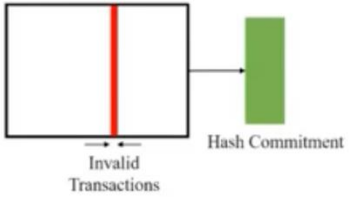
- Pushes hash commitment to the trusted blockchain
- Full block not available to other side blockchain nodes
- The invalid block becomes part of the transaction ordering in the trusted blockchain

Data Availability Attack on Silde Blockchain

Side Blockchains with a majority of dishonest nodes are vulnerable to data availability attacks [Sheng '20]



Adversary creates an invalid block



Note: DA attack cannot occur when side blockchains have a majority of honest nodes → majority vote on " is something missing "

Adversarial Side blockchain node:

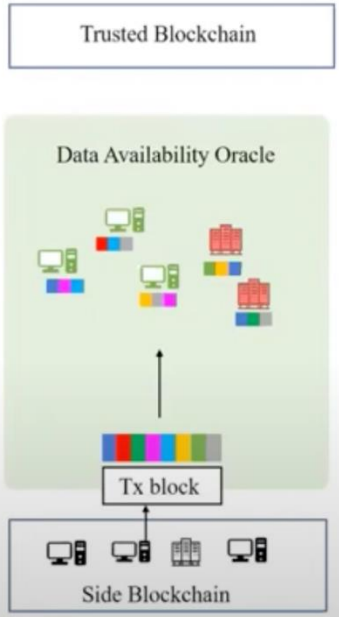
- Pushes hash commitment to the trusted blockchain
- Full block not available to other side blockchain nodes
- The invalid block becomes part of the transaction ordering in the trusted blockchain

Using a Data Availability Oracle

An oracle layer was introduced to ensure data availability [Sheng '20]

Using a Data Availability Oracle

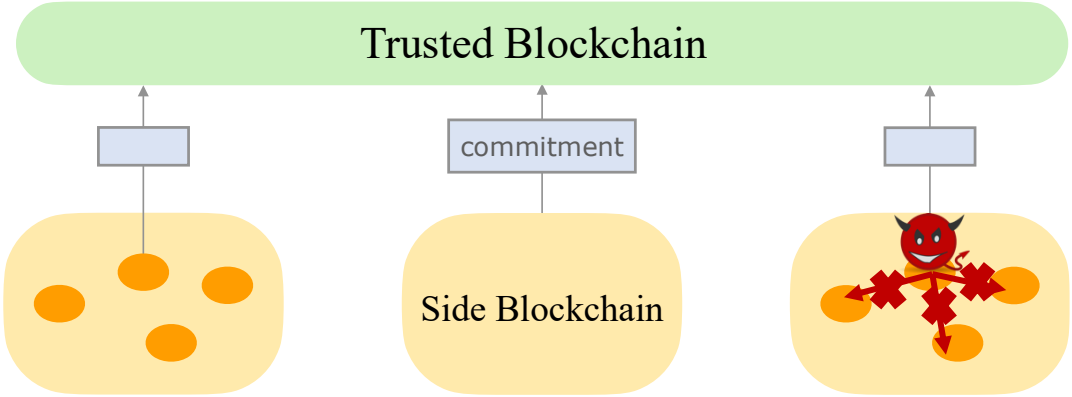
An oracle layer was introduced to ensure data availability [Sheng '20]



Oracle layer goal

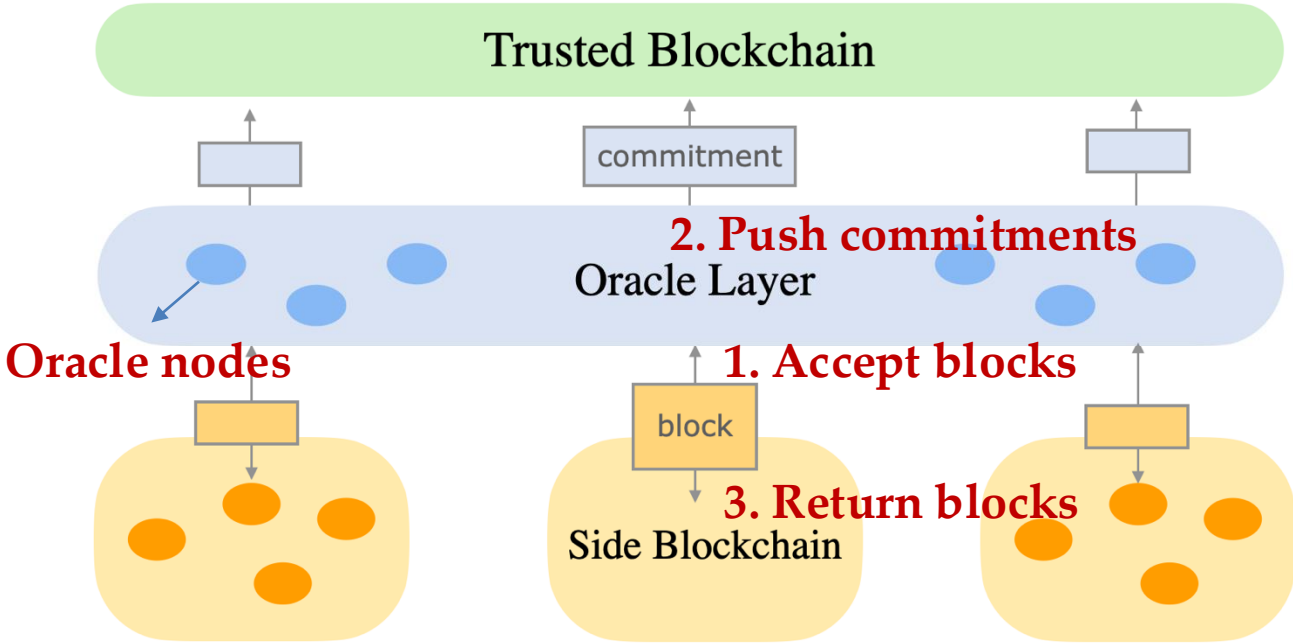
- Accept a Tx block
- Collectively and efficiently store chunks of the Tx block (to guarantee availability)
- Push the Tx block's hash commitment iff the block is available
- Oracle nodes can be malicious (honest majority)

Data Availability Attack

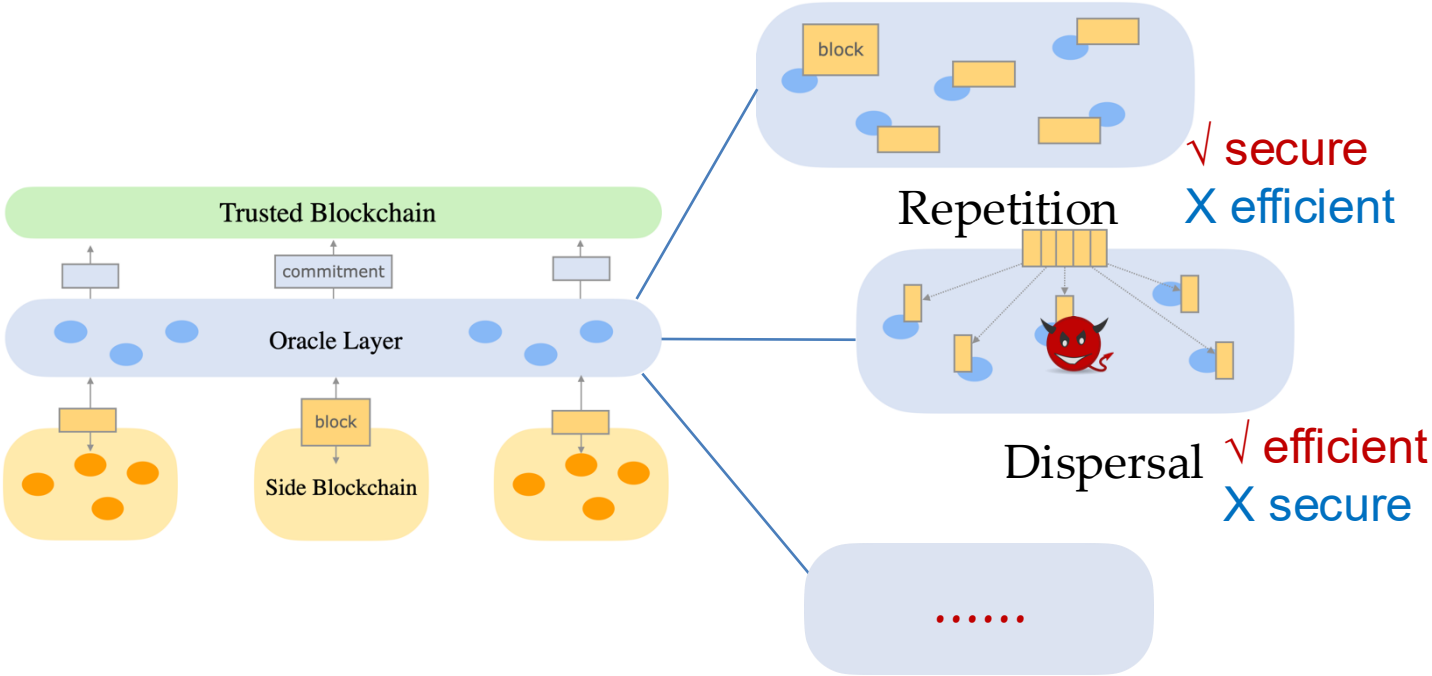


- Side Blockchains
  - Order of blocks ← order of hashes
  - ***Attack: not transmit the block to others***

Data Availability Oracle

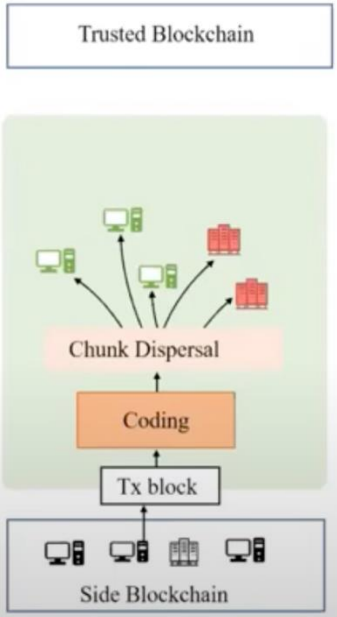


Repetition and Dispersal

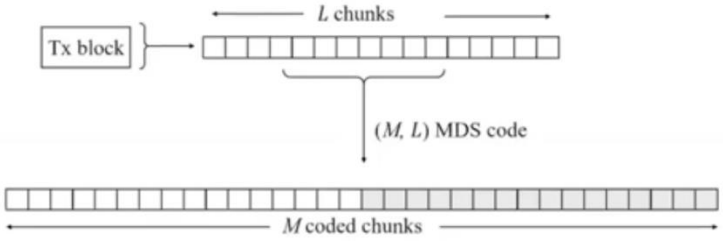


Using a Data Availability Oracle

An oracle layer was introduced to ensure data availability [Sheng '20]



- Transaction block: chunked and coded
- Coded chunks dispersed among oracle nodes



For MDS codes, iff at least L coded chunks are present among honest oracle layer nodes $\rightarrow$ block availability is guaranteed

Resources

- ECE/COS 470, Pramod Viswanath, Princeton 2024